Instituto Federal de Educação, Ciência e Tecnologia

Triângulo Mineiro – Campus Uberlândia Centro

Curso Superior de Tecnologia em Sistemas para Internet

Projeto Back-End Monolítico com ORM

Slide 3 – APIs REST - Introdução

Prof. Carlos Eduardo de Carvalho Dantas

carloseduardodantas@iftm.edu.br

https://carloseduardoxp.github.io/

Introdução

- E se amanhã o sistema demandar:
  - App Android
  - App iOS
  - Front-end React
  - Sistema externo consumindo a API

- O servidor não deveria retornar HTML, porque:
  - Android não usa HTML
  - iOS não usa HTML
  - Outro sistema quer dados, não interface.

```
<html>
 <body>
 <form action = "/cadastrar" method = "post" >
 <input type = "text" name = "nome" />
 <input type = "text" name = "dataNascimento" />
 <input type = "submit" value = "Cadastrar" />
 </form>
 </body>
</html>
```

Web Services usando Protocolo SOAP

- Uma alternativa seria usar Web Services (WS-*) baseadas no protocolo SOAP que trafega dados usando XML.
- Com SOAP é possível ter interoperabilidade, isto é, a capacidade de diferentes plataformas e linguagens consumirem o serviço da mesma forma, desde que sigam o protocolo.

Request →

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:cad="http://exemplo.org/cadastro">
   <soapenv:Header/>
   <soapenv:Body>
      <cad:SalvarPessoa>
         <cad:Nome>João da Silva</cad:Nome>
         <cad:DataNascimento>1985-06-20</cad:DataNascimento>
      </cad:SalvarPessoa>
   </soapenv:Body>
</soapenv:Envelope>
```

← **Response**

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:cad="http://exemplo.org/cadastro">
   <soapenv:Header/>
   <soapenv:Body>
      <cad:SalvarPessoaResponse>
         <cad:Mensagem>Pessoa cadastrada com sucesso</cad:Mensagem>
         <cad:IdPessoa>123</cad:IdPessoa>
      </cad:SalvarPessoaResponse>
   </soapenv:Body>
</soapenv:Envelope>
```

INSTITUTO FEDERAL TRIÂNGULO MINEIRO Campus Uberlândia Centro

Porém ...

- Mesmo operações simples exigem muita "burocracia" de XML.
- Envolve o conhecimento de WSDL, XML Schema, namespaces, e padrões como WS-Security, WS-ReliableMessaging, entre outros.
- Frequentemente é necessário usar ferramentas para gerar código a partir do WSDL (como wsimport em Java).
- Mensagens SOAP são mais pesadas, devido à verbosidade do XML e à inclusão de cabeçalhos adicionais.
- SOAP normalmente ignora os métodos HTTP (GET, POST, etc.) como semântica — ele encapsula tudo em um POST
- SOAP não é bem suportado diretamente por navegadores.

REST

- O método HTTP POST é comumente usado para criar um novo recurso.
- O corpo da requisição costuma ser enviado em JSON, que é mais leve que XML e amplamente suportado.
- O servidor responde com um código de status HTTP padrão (201 Created), e muitas vezes inclui o local do novo recurso criado (Location).
- Esse tipo de serviço pode ser consumido facilmente por aplicações web, mobile, scripts Python, front-ends JavaScript, etc.

Request →

```http
Content-Type: application/json
{
  "nome": "João da Silva",
  "dataNascimento": "1985-06-20"
}
```

← Response

```http
Location: /api/pessoas/123
Content-Type: application/json
{
  "id": 123,
  "mensagem": "Pessoa cadastrada com sucesso"
}
```

INSTITUTO FEDERAL TRIÂNGULO MINEIRO Campus Uberlândia Centro

REST - Características

- Uso adequado dos métodos HTTP (GET, POST, PUT, DELETE, etc.);
- Uso apropriado de URLs para identificar recursos;
- Utilização de códigos de status HTTP padronizados para indicar sucesso ou erro (ex: 200 OK, 404 Not Found, 201 Created);
- Uso eficiente de cabeçalhos HTTP (como Content-Type, Accept, Authorization);
- Hiperligações entre recursos, promovendo a navegabilidade entre diferentes partes da API (conceito conhecido como HATEOAS).

REST - Recursos

- São as entidades do domínio da aplicação que a API expõe e manipula.
  - Eles representam dados ou objetos do domínio da aplicação, como *usuários, produtos, pedidos, livros, artigos*, etc.
  - Cada recurso tem um endereço único, por exemplo:

    GET /clientes/123 → Recurso do cliente com ID 123. No caso, o endereço para acessar o ID 123 do cliente é chamado de URI (Uniform Resource Identifier)

    Recurso: O cliente com ID 123

    URI: /clientes/123
  - Os recursos não têm "ações" embutidas — quem faz a ação são os métodos (GET, POST, etc).
  - A forma mais comum de representar um recurso é via JSON (ou XML), com os atributos daquele objeto.

Métodos HTTP em APIs REST

- Os métodos HTTP indicam a ação que será realizada sobre um recurso.

Método HTTP	Uso Principal	Exemplo	Idempotência
GET	Recuperar um recurso	GET /clientes/123	sempre igual
POST	Criar um novo Recurso	POST /clientes	repetir não muda
PUT	Atualizar um recurso existente	PUT /clientes/123	repetir não muda
PATCH	Atualizar parcialmente um recurso	PATCH /clientes/123	repetir não muda
DELETE	Remover um recurso	DELETE /clientes/123	cria outro

Métodos HTTP em APIs REST

- Métodos GET são enviados naturalmente pelo navegador, como por exemplo, ao digitar http://www.globo.com

- Chamadas POST são feitas via formulário

```
<form method="get">
<form method="post">
```

- Chamadas PUT, DELETE ou PATCH podem ser chamadas via Javascript ou outra linguagem de Programação

```
fetch("/clientes/123", {
  method: "DELETE"
});
```

```
dados = {
    "nome": "Carlos Eduardo",
    "email": "carlos@email.com"
}

response = requests.put(url, json=dados)
```

- Ferramentas externas como POSTMAN ou CURL podem enviar qualquer método

Métodos HTTP em APIs REST - exemplos

Método HTTP	Recurso	URI Específica	Ação
GET	clientes	/clientes	Listar todos os clientes
GET	clientes	/clientes/1	Buscar os detalhes do cliente de ID = 1
GET	usuarios	/usuarios?nome=joao	Buscar os usuários em que o atributo nome possua o texto “joao”
GET	filmes	/filmes?ano=1994&genero=drama	Busca os filmes de 1994 do gênero drama
POST	clientes	/clientes	Criar um novo cliente
PUT	clientes	/clientes/1	Atualiza o cliente de ID = 1
DELETE	clientes	/clientes/1	Exclui o cliente de ID = 1

REST – Recursos - Convenções

- Use substantivos no plural e sem caracteres especiais (por ex: acentos)
  - clientes, pedidos, produtos, categorias, usuarios.
- Evite verbos nas URIs
  - A ação é indicada pelo **método HTTP** (GET, POST, PUT, DELETE), não pela URI.
  - **ERRADO**: GET /buscarClientes
  - **CERTO**: GET /clientes?nome=joao
- Use letras minúsculas e hífens para separar palavras
  - GET /historico-pedidos
- Organize com hierarquia quando houver sub-recursos
  - GET /clientes/42/pedidos - Retorna a coleção de pedidos do cliente 42.

REST – Recursos - Convenções

- **/clientes** – abordagem comum em projetos pequenos, APIs internas, exemplos acadêmicos e microserviços simples

- **/api/clientes** – a idéia é separar API de páginas web. Isso é comum em aplicações que têm: backend REST, frontend web tradicional, frontend SPA (React, Angular)
  - /site/home
    /site/login
    /api/contatos
    /api/produtos

- **/v1/clientes** – a idéia é versionar a API. Se mudar o contrato da API, pode-se criar uma nova versão sem quebrar clientes antigos

REST – Recursos – Spring Boot

1) Em um **@RestController**, cada recurso da API normalmente possui um caminho base, como **@RequestMapping("/contatos")** para o recurso "contatos".

```java
13 @RestController
14 @RequestMapping("/contatos")
15 public class ContatoController {
16
17     private List<Contato> contatos = new ArrayList<>(List.of(
18                                 new Contato(codigo: 1, nome: "Goku"),
19                                 new Contato(codigo: 2, nome: "Vegeta"))
20         );
21
22     @GetMapping
23     public List<Contato> listar() {
24         return contatos;
25     }
26 }
```

O caminho base do recurso já está mapeado para todos os métodos da classe

```java
12 @RestController
13 public class ContatoController {
14
15     private List<Contato> contatos = new ArrayList<>(List.of(
16                                 new Contato(codigo: 1, nome: "Goku"),
17                                 new Contato(codigo: 2, nome: "Vegeta"))
18         );
19
20     @GetMapping("/contatos")
21     public List<Contato> listar() {
22         return contatos;
23     }
24 }
```

Todos os métodos precisarão mapear o caminho base, o que é mais propenso a erros.

REST – Códigos de Retorno – Spring Boot

2) Para manter consistência, controle de status HTTP e possibilidade de customizar cabeçalhos, é recomendado padronizar os retornos dos métodos dos controllers usando **ResponseEntity<?>**, em vez de retornar diretamente objetos, listas ou strings.

```java
16  @RequestMapping("/contatos")
17  public class ContatoController {
18
19      private List<Contato> contatos = new ArrayList<>(List.of(
20                                  new Contato(codigo: 1, nome: "Goku"),
21                                  new Contato(codigo: 2, nome: "Vegeta"))
22          );
23
24      @GetMapping
25      public ResponseEntity<List<Contato>> listar() {
26          return ResponseEntity.ok(contatos);
27      }
28
29      @GetMapping("/{id}")
30      public ResponseEntity<Contato> buscarPorId(@PathVariable Integer id) {
31          for (Contato contato: contatos) {
32              if (contato.getCodigo().equals(id)) {
33                  return ResponseEntity.ok(contato);
34              }
35          }
36          return ResponseEntity.notFound().build();
37      }
38  }
```

Se não existir o id, retorna 404 (recurso não existe)

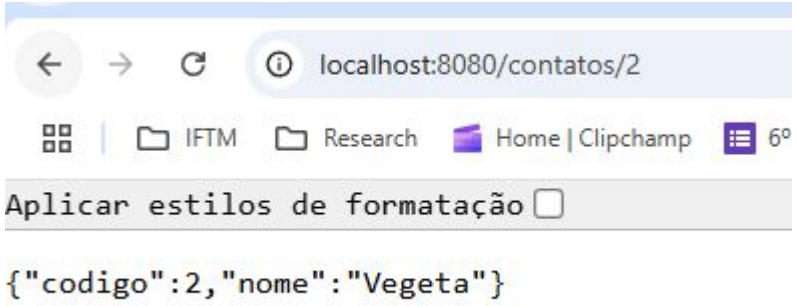
```java
16  @RequestMapping("/contatos")
17  public class ContatoController {
18
19      private List<Contato> contatos = new ArrayList<>(List.of(
20                                  new Contato(codigo: 1, nome: "Goku"),
21                                  new Contato(codigo: 2, nome: "Vegeta"))
22          );
23
24      @GetMapping
25      public ResponseEntity<List<Contato>> listar() {
26          return ResponseEntity.ok(contatos);
27      }
28
29      @GetMapping("/{id}")
30      public Contato buscarPorId(@PathVariable Integer id) {
31          for (Contato contato: contatos) {
32              if (contato.getCodigo().equals(id)) {
33                  return contato;
34              }
35          }
36          return null;
37      }
```

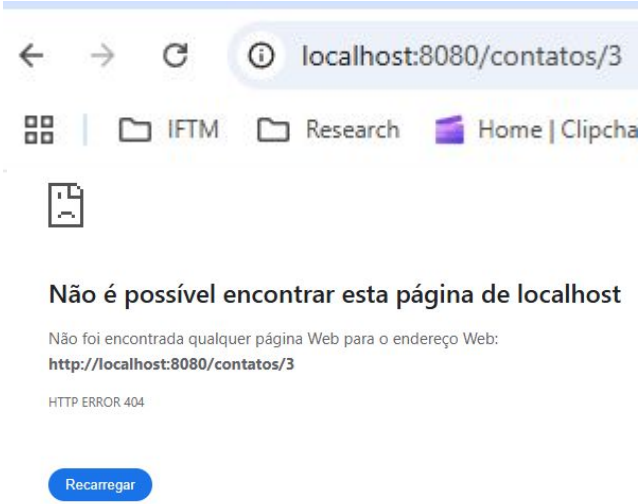
Retornar null ou uma exceção EntityNotFoundException não é o ideal, pois o recurso não existe

REST – Códigos de Retorno

Recurso /contatos/2 – Retorno 200

Recurso /contatos/3 – Retorno 404





REST – Recursos – Spring Boot

3) Lembrando que em REST, cada recurso é único.

```
@GetMapping
public ResponseEntity<List<Contato>> listar() {
    return ResponseEntity.ok(contatos);
}

@GetMapping
public ResponseEntity<List<Contato>> buscar() {
    return ResponseEntity.ok(contatos);
}
```

Não irá gerar erro de compilação porque os métodos possuem nomes diferentes

Mas irá gerar erro ao subir o Spring Boot porque ambos mapeiam o mesmo recurso GET /contatos

```
br.edu.iftm.tspi.pbackorm.contato.controller.ContatoController#buscar()
to {GET [/contatos]}: There is already 'contatoController' bean method
br.edu.iftm.tspi.pbackorm.contato.controller.ContatoController#listar() mapped.
```

INSTITUTO FEDERAL
TRIÂNGULO MINEIRO
Campus Uberlândia Centro

REST – Códigos de Retorno

HTTP	Situação	Explicação	Spring Boot
200 OK	Sucesso (buscar/listar)	Operação deu certo	ResponseEntity.ok(obj); (com recurso no body) ResponseEntity.ok().build(); (sem retorno)
201 Created	Criou um recurso	Recurso foi criado com sucesso	ResponseEntity.created(uri).body(obj); ResponseEntity.status(HttpStatus.CREATED).body (obj);
204 No Content	Sucesso sem conteúdo (delete/update)	Deu certo, mas não há body	ResponseEntity.noContent().build();
400 Bad Request	Requisição inválida	Validação falhou	ResponseEntity.badRequest().body(erro);
401 Unauthorized	Não autenticado	Falta login/token válido	ResponseEntity.status(HttpStatus.UNAUTHORIZE D).build();

INSTITUTO FEDERAL
TRIÂNGULO MINEIRO
Campus Uberlândia Centro

REST – Códigos de Retorno

HTTP	Situação	Explicação	Spring Boot
403 Forbidden	Autenticado, mas sem permissão	Sem acesso	ResponseEntity.status(HttpStatus.FORBIDDEN).build();
404 Not Found	Recurso não existe	Não encontrado	ResponseEntity.notFound().build();
409 Conflict	Conflito de estado	duplicado / regra de negócio impede	ResponseEntity.status(HttpStatus.CONFLICT).body(erro);
422 Unprocessable Entity	Requisição inválida	Entidade/processamento inválido	ResponseEntity.unprocessableEntity().body(erro);
500 Internal Server Error	Exceção não tratada	Erro inesperado no servidor	ResponseEntity.internalServerError().body(erro);

REST – Códigos de Retorno

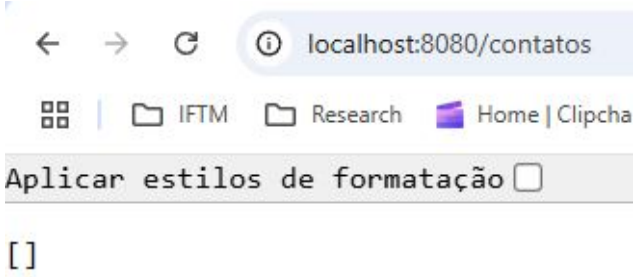
Pergunta: Se o método listar() retornar uma lista vazia, o que é mais adequado: retornar 200 ou 404?

Resposta: O correto é retornar 200 OK com a lista vazia.

Por que? O recurso solicitado (**/contatos**) existe, mas não há elementos no momento.

404 Not Found só deve ser usado quando o recurso em si não existe, como no caso de

GET /contatos/3 - Se não houver contato com ID = 3, aí sim cabe o 404.

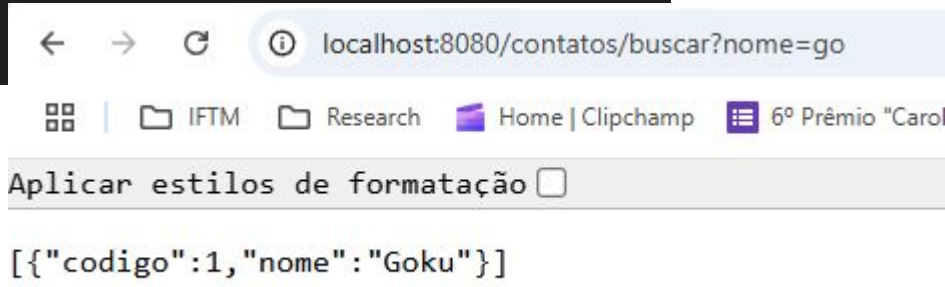


REST – Filtros vs Recursos – Spring Boot

4) Use @PathVariable quando o valor for um identificador do recurso.

Use @RequestParam quando o valor for um filtro, campo de busca ou critério opcional.

```java
@GetMapping("/buscar")
public ResponseEntity<List<Contato>> buscarPorNome(@RequestParam(required = true) String nome) {
    List<Contato> resposta = new ArrayList<>();
    for (Contato contato: contatos) {
        if (contato.getNome().toLowerCase().contains(nome)) {
            resposta.add(contato);
        }
    }
    return ResponseEntity.ok(resposta);
}
```



Criamos uma nova rota “/contatos/buscar” porque a rota “/contatos” já está sendo usada para listar todos

REST – Filtros vs Recursos

@PathVariable
- Identifica um recurso específico

Recurso = /clientes
Recurso específico = /clientes/42
Sub-recurso = /clientes/42/pedidos

@RequestParam
- Filtra ou refina a busca

Recurso = /clientes
Filtro - /clientes?nome=joao

REST – Filtros

5) O que é mais adequado em uma API REST: criar uma rota separada como /contatos/buscar, ou reutilizar a rota /contatos e permitir a busca através de parâmetros opcionais?

```
@GetMapping()
public ResponseEntity<List<Contato>> buscar(@RequestParam(required = false) String nome) {
    if (nome == null) {
        return ResponseEntity.ok(contatos);
    }
    List<Contato> resposta = new ArrayList<>();
    for (Contato contato: contatos) {
        if (contato.getNome().toLowerCase().contains(nome.toLowerCase())) {
            resposta.add(contato);
        }
    }
    return ResponseEntity.ok(resposta);
}
```

Se o parâmetro nome for informado, a API retorna apenas os contatos que correspondem ao filtro. Caso o parâmetro não seja enviado, a resposta trará todos os contatos.

REST – Filtros

4) Neste caso onde temos poucos campos de texto, o ideal é reutilizar a rota e colocar os parâmetros como opcionais.

Desta forma, o método **listar()** deixará de existir, e o método **buscar()** fará tanto a função de listar quanto de buscar por parâmetros.

```
GET /contatos          → Lista todos os contatos
GET /contatos?nome=ana → Lista contatos cujo nome contém "ana"
```

REST prefere substantivos, não ações. A URI deve apresentar o recurso, não a operação.

GET /contatos?nome=joao

GET /contatos/buscar?nome=joao

REST – Exemplos URLs

4) Exemplos:

4.1) Filtros:

GET /produtos?categoria=eletronicos

GET /usuarios?cidade=uberlandia

GET /pedidos?status=pago

4.2) Paginação:

GET /clientes?page=2&size=10 (página 2 com 10 registros)

4.3) Ordenação:

GET /produtos?sort=preco,desc (ordenado por preço)

4.4) Combinando tudo:

GET /produtos?categoria=eletronicos&page=0&size=10&sort=preco,desc

REST – Métodos HTTP

5) Padronizar o uso dos métodos HTTP para sua respectiva finalidade:

GET: leitura sem efeitos colaterais (idempotente)

POST: criação de recursos

PUT vs **PATCH**: atualização total ou parcial

DELETE: remoção

REST – Criação de Recurso – Spring Boot

E como cria um recurso?

```
@PostMapping
public ResponseEntity<Contato> novo(@RequestBody Contato contato) {
    contatos.add(contato);
    return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(contato);
}
```

Receberá um json com os dados do contato, e irá converter para um objeto Java do tipo Contato.

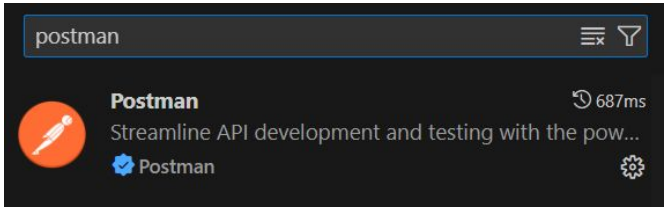
HttpStatus.CREATED significa que o servidor irá devolver o código de resposta 201, isto é, um novo recurso foi criado no servidor.

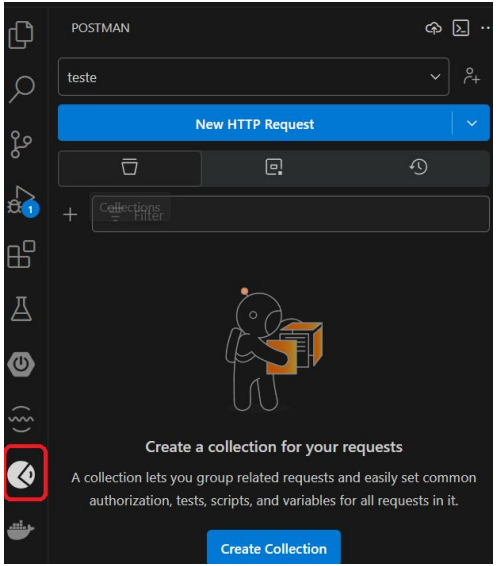
.body(contato) irá devolver o objeto. Caso tenha um campo de auto-incremento, o cliente receberá o objeto preenchido com o valor

REST – Postman – Spring Boot

E como consome o recurso se o navegador só aceita requisições do tipo GET?

Podemos instalar o plugin do **Postman**



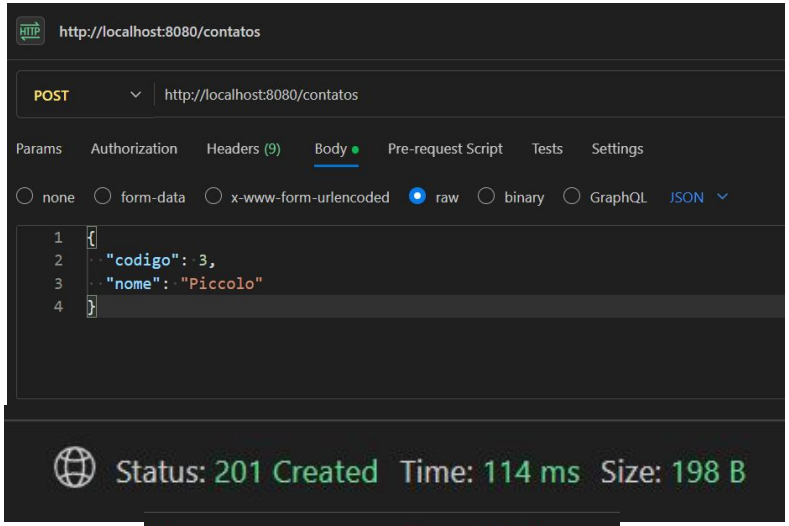


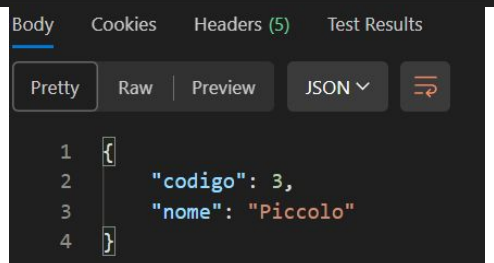
REST – Postman – Spring Boot

1) Clique em “New HTTP Request”
2) Selecione POST como método a ser usado
3) Digite a URL: http://localhost:8080/contatos
4) Em body, selecione “raw” e o tipo JSON
5) Coloque o json no corpo da requisição

```
{
  "codigo": 3,
  "nome": "Piccolo"
}
```

6) Clique em “Send” para enviar a requisição
7) Veja o retorno 201 e o json com o objeto criado.





REST – Conflito de Recursos – Spring Boot

E se o objeto já existir? No caso, tentar salvar com uma chave primária que já está no banco?

Precisamos devolver um erro 409 de conflito.

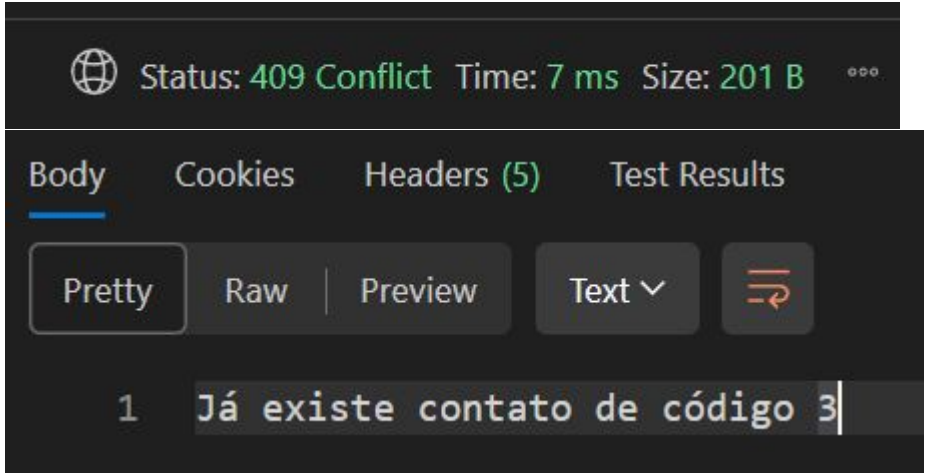
```
@PostMapping
public ResponseEntity<?> novo(@RequestBody Contato novoContato) {
    boolean existe = contatos.stream()
                    .anyMatch(contato -> contato.getCodigo().equals(novoContato.getCodigo()));

    if (existe) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body("Já existe contato de código "+novoContato.getCodigo());
    }
    contatos.add(novoContato);
    return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(novoContato);
}
```

REST – Conflito de Recursos – Spring Boot

Funcionar funciona.

Mas .... deveria ser padrão retornarmos objetos Json. Como faz?



REST – Construindo DTOs – Spring Boot

Crie um pacote chamado dto e uma classe ErroDTO.java.

```
@Data
@AllArgsConstructor
public class ErroDTO {

    private String mensagem;

    private LocalDateTime data;

}
```

REST – Criação de Recurso com DTO – Spring Boot

Classe atualizada

```
@PostMapping
public ResponseEntity<?> novo(@RequestBody Contato novoContato) {
    boolean existe = contatos.stream()
                .anyMatch(contato -> contato.getCodigo().equals(novoContato.getCodigo()));
    if (existe) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
        .body(new ErroDTO("Já existe contato de código "+novoContato.getCodigo()
                        ,LocalDateTime.now()));
    }
    contatos.add(novoContato);
    return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(novoContato);
}
```

REST – Mensagem de Retorno DTO – Spring Boot

Parece mais adequado

Body Cookies Headers (5) Test Results Status: 409 Conflict Time: 11 ms Size: 257 B

Pretty Raw Preview JSON


```
1 {"mensagem":"Já existe contato de código 3","dataHora":"2026-03-20T20:11:45.1756099"}
```

REST – DTOs

Ok, o que é um DTO?

- DTO significa Data Transfer Object (Objeto de Transferência de Dados).
- É uma classe simples usada para transportar dados entre camadas da aplicação (por exemplo, entre o backend e o frontend), sem expor diretamente as entidades do banco de dados.

REST – Validação campos – Spring Boot

E se enviarem um nome de contato vazio?

```
@PostMapping
public ResponseEntity<?> novo(@RequestBody Contato novoContato) {
    boolean existe = contatos.stream()
                .anyMatch(contato -> contato.getCodigo().equals(novoContato.getCodigo()));
    if (existe) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
        .body(new ErroDTO("Já existe contato de código "+novoContato.getCodigo()
                        ,LocalDateTime.now()));
    }
    if (novoContato.getNome() == null || novoContato.getNome().isBlank()) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErroDTO(mensagem: "Contato com nome vazio",LocalDateTime.now()));
    }
    contatos.add(novoContato);
    return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(novoContato);
}
```

REST – Mensagem DTO – Spring Boot

Body Cookies Headers (4) Test Results

Status: 400 Bad Request Time: 8 ms Size: 222 B

Pretty Raw Preview


```
1 {"mensagem":"Contato com nome vazio","dataHora":"2026-03-20T20:26:59.9645049"}
```

REST – Atualizando Recurso – Spring Boot

E como atualiza um recurso?

```
@PutMapping("/{id}")
public ResponseEntity<?> atualizar(@PathVariable Integer id,
                                   @RequestBody Contato contatoAtualizado) {
    for (Contato contato: contatos) {
        if (contato.getCodigo().equals(id)) {
            contato.setNome(contatoAtualizado.getNome());
            return ResponseEntity.ok(contatoAtualizado);
        }
    }
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(
        new ErroDTO("Contato não encontrado "+id,LocalDateTime.now()));
}
```

Se tentarmos atualizar um contato que não existe, retornará 404

REST – Atualizando Recurso com sucesso – Spring Boot

http://localhost:8080/contatos/3

PUT http://localhost:8080/contatos/3

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "codigo": 3,
3   "nome": "Piccolo fundido com o Kamisama"
4 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
1 {
2     "codigo": 3,
3     "nome": "Piccolo fundido com o Kamisama"
4 }
```

REST – Falha ao atualizar recurso – Spring Boot

PUT http://localhost:8080/contatos/10

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

```
1 {
2     "codigo": 10,
3     "nome": "Bulma"
4 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
1 {
2     "mensagem": "Contato não encontrado 10",
3     "dataHora": "2026-03-20T21:10:12.4552763"
4 }
```

Status: 404 Not Found Time: 14 ms Size: 130 B

INSTITUTO FEDERAL TRIÂNGULO MINEIRO Campus Uberlândia Centro

REST – Atualizar Recursos

Pergunta: Se o ID já está dentro do objeto JSON, por que também preciso colocá-lo na URL quando uso PUT?

É uma boa prática (e muitas vezes exigida por frameworks) para garantir integridade e evitar inconsistências.

No POST não conseguimos colocar o ID na URL porque o recurso ainda não existe.

REST – PUT vs PATCH

- PUT (Atualização total) **@PutMapping**
  - Substitui todo o recurso
  - Todos os campos devem ser enviados
  - Campos não enviados podem ser sobrescritos (ex: null)
  - É idempotente

```
{
"nome": "Carlos",
"email": "carlos@email.com"
}
```

- PATCH (Atualização parcial) **@PatchMapping**
  - Atualiza apenas alguns campos
  - Envia somente o que deseja alterar
  - Não altera os demais campos
  - Também é idempotente (na prática)

```
{
"email": "carlos@email.com"
}
```

REST – Excluir Recurso – Spring Boot

E como exclui um recurso?

```
@DeleteMapping("/{id}")
public ResponseEntity<?> deletar(@PathVariable Integer id) {
    boolean removido = contatos.removeIf(
        contato -> contato.getCodigo().equals(id));
    if (removido) {
        return ResponseEntity.noContent().build();
    }
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(
        new ErroDTO("Contato não encontrado "+id, LocalDateTime.now()));
}
```

O retorno é 204 porque o recurso foi excluído, logo não tem conteúdo para ser retornado

INSTITUTO FEDERAL
TRIÂNGULO MINEIRO
Campus Uberlândia Centro

REST – Códigos de retorno

Código	Método	Situação	Ação
200	GET,PUT	requisição bem-sucedida	Todos os métodos de busca
201	POST	requisição bem-sucedida	Cria um novo objeto
204	DELETE	requisição bem-sucedida	Objeto excluído, sem conteúdo de retorno
400	POST,PUT	erro na requisição	Objeto com valores inválidos
404	GET,PUT,DELETE	erro na requisição	Recurso não encontrado
409	POST	erro na requisição	Violação de regra de unicidade

INSTITUTO FEDERAL TRIÂNGULO MINEIRO Campus Uberlândia Centro

Referências Bibliográficas

- MICROSOFT. *REST API Design Guidelines*. Microsoft Learn. Disponível em: https://learn.microsoft.com/azure/architecture/best-practices/api-design. Acesso em: 5 mar. 2026.

- VMWARE. *Spring Boot Reference Documentation*. Disponível em: https://docs.spring.io/spring-boot/docs/current/reference/html/. Acesso em: 5 mar. 2026.

- SAUDATE, Alexandre. *REST: Construa APIs inteligentes de maneira simples*. São Paulo: Casa do Código, 2013.